

Project Setup, Hibernate, Flyway, and GitHub Workflow Documentation

Project Setup, Hibernate, Flyway, and GitHub Workflow Documentation

This document consolidates our discussion into a structured guide you can study and reference.

1. Docker Compose and Environment Variables

Key Points

- .env file values are injected by Docker Compose into containers.
- Spring Boot does **not** automatically read .env files when running locally (e.g., in IntelliJ).
- In application.properties, placeholders like \${DB_NAME:carpool} mean:
 - Use DB_NAME if available.
 - Otherwise, default to carpool.

Fix

- Ensure IntelliJ run configurations include environment variables (DB_NAME, DB_HOST, etc.).
 - Or create a local profile (application-local.properties) with hardcoded values.
-

2. Hibernate Column Type Mismatches

Issue

- Entity field type vs. database column type mismatch.
- Example: Integer seatsReserved vs. MySQL TINYINT column.

Solutions

- **Option 1:** Match entity to DB → use Byte or Short in Java.
- **Option 2:** Match DB to entity → alter column to INT or SMALLINT.

Java ↔ SQL Mapping

- byte → TINYINT

- short → SMALLINT
- int → INTEGER

Arithmetic Promotion

- Arithmetic on byte or short → promoted to int.
- Must cast back:

```
short updatedSeats = (short) (ride.getAvailableSeats() - request.seatsReserved());
```

3. Flyway Migration Management

Rules

- Migration files are immutable once applied.
- Editing an applied migration causes checksum mismatch.

Correct Workflow

1. Create new migration file (e.g., V3__alter_bookings.sql).
2. Apply changes via flyway migrate.
3. If a migration fails:
 - Manually fix partial DB changes.
 - Run flyway repair to clear failed state.
 - Create a new migration for the fix.

Failed Migration Example

- If V4 fails:
 - Clean up DB.
 - Run flyway repair.
 - Add V5__fix.sql with corrected SQL.

4. GitHub Workflow in IntelliJ

Initial Setup

1. Create repo on GitHub.
2. Initialize Git locally:

```
git init
git add .
git commit -m "Initial commit"
```

3. Link remote:

```
git remote add origin https://github.com/<username>/<repo>.git
git branch -M main
git push -u origin main
```

IntelliJ GUI Path

- **VCS menu** → *Import into Version Control* → *Share Project on GitHub*.
- Log in to GitHub.
- IntelliJ handles commit and push.

Checking Account

- **File** → **Settings** → **Version Control** → **GitHub** shows connected accounts.
- Or run `git remote -v` in terminal to see which repo URL is linked.

Switching Accounts

1. Update Git config:

```
git config --global user.name "NewUsername"
git config --global user.email "newemail@example.com"
```

Or local-only:

```
git config user.name "NewUsername"
git config user.email "newemail@example.com"
```

2. Update remote:

```
git remote set-url origin https://github.com/NewAccount/repo.git
```

3. Push again.

Rewriting Commit Author

- To fix initial commit author:

```
git commit --amend --reset-author
git push --force origin main
```

-  Only safe if repo is new and no collaborators yet.

5. Summary of Best Practices

- Keep DB and entity types consistent.
- Use Flyway with append-only migrations.

- Always repair failed migrations before continuing.
- Configure IntelliJ to use correct GitHub account.
- Use git config to set author identity properly.
- Avoid editing old migrations or force-pushing in shared repos.

This documentation should serve as a reference for setting up Docker, handling Hibernate type mismatches, managing Flyway migrations, and pushing projects to GitHub with the correct account identity.